



Projeto: Sistema de Doação de Alimentos

Integrantes:

- Enzo Ponte Gamberi - RA: 10389931
- João Guilherme Messias de Oliveira Santos – RA: 10426110
- Thiago Ruiz Fernandes Silva – RA: 10426057

Professor: Luiz Carlos Machi Lozano

SUMÁRIO

1. Introdução
2. Definição da demanda
 - 2.1. O problema ou oportunidade percebida
 - 2.2. A razão ou justificativa para esta demanda
 - 2.3. Descrição sucinta do produto de software
 - 2.4. Clientes, usuários e demais envolvidos/impactados
 - 2.5. Principais etapas necessárias para construir o produto
 - 2.6. Principais critérios de qualidade para o produto
3. Requisitos do Produto
4. Wireframes
5. Modelagem
 - 5.1. Diagrama de Casos de Uso



5.2. Diagrama de Domínio/Classe

5.3. Diagrama de Sequência

6. Descrição da Arquitetura do Sistema e Ferramentas

6.1. Camada de Front-End

6.2. Camada de Back-End

6.3. Banco de Dados

6.4. Ferramentas de Desenvolvimento e DevOps

6.5. Infraestrutura em Nuvem (AWS)

1. INTRODUÇÃO

Entregar um produto de software no prazo, respeitando o orçamento e garantindo alta qualidade são os objetivos fundamentais da engenharia de software. Para alcançar esses objetivos, é imprescindível a aplicação de ferramentas e técnicas estruturadas que evitem o desenvolvimento inadequado do sistema. O presente projeto, desenvolvido no âmbito da disciplina de Laboratório de Engenharia de Software, visa colocar em prática esses recursos por meio da proposição e desenvolvimento de um sistema completo e funcional.

O escopo escolhido para este desafio é o desenvolvimento de um **Sistema de Doação de Alimentos**, uma plataforma digital orientada à web com forte caráter extensionista, projetada para solucionar um problema de cenário real. O cenário atual apresenta um paradoxo crítico: altos índices de desperdício de alimentos viáveis para consumo por parte do setor de comércio e serviços, em contraste com a insegurança alimentar enfrentada por parcelas vulneráveis da população.

Dessa forma, o sistema proposto atua como uma ponte tecnológica, estabelecendo uma rede de comunicação eficiente e rastreável entre doadores (pessoas ou empresas) e instituições receptoras. Este documento atuará como um artefato vivo que evoluirá ao longo de todo o ciclo de vida do software. Serão detalhadas as etapas de engenharia de requisitos, modelagem de sistema, definição da arquitetura, até a sua futura implantação em ambiente de nuvem, refletindo de forma progressiva o avanço do desenvolvimento.



2. DEFINIÇÃO DA DEMANDA

2.1 O problema ou oportunidade percebida

A raiz do problema que este software se propõe a resolver reside na extrema ineficiência logística e na ausência de canais de comunicação centralizados para doações de perecíveis e não-perecíveis. Diariamente, estabelecimentos comerciais (como padarias, restaurantes e mercados) e pessoas físicas descartam volumes significativos de alimentos em perfeitas condições sanitárias. O descarte ocorre, em grande parte, porque não há um método ágil e confiável para anunciar essas sobras e repassá-las a quem necessita antes de seu vencimento.

2.2 A razão ou justificativa para esta demanda

A justificativa para a construção desta plataforma apoia-se na responsabilidade socioambiental e na eficiência tecnológica. Uma plataforma digital dedicada reduz o atrito da comunicação e centraliza a informação. Isso elimina o esforço de organizações não governamentais (ONGs) que gastam recursos escassos buscando doações de forma não estruturada. O sistema mitigará o risco de viagens perdidas por parte das instituições, garantindo o escoamento rápido dos perecíveis e reduzindo drasticamente o volume de lixo orgânico.

2.3 Descrição sucinta do produto de software

O produto final será uma aplicação web baseada em uma arquitetura cliente-servidor. O sistema gerenciará dois perfis principais de acesso: Doadores e Instituições. O módulo do doador permitirá o cadastro rápido de alimentos disponíveis. O módulo da Instituição fornecerá um *feed* interativo, com capacidades de filtragem por categoria de alimento, além de um mecanismo transacional simples para reserva exclusiva da doação, bloqueando-a para outros usuários e atualizando seu status em tempo real.

2.4 Clientes, usuários e demais envolvidos/impactados

- **Usuários Doadores:** Pessoas físicas da comunidade dispostas a doar excedentes, bem como entidades jurídicas de pequeno a médio porte (padarias, hortifrutis e restaurantes locais).
- **Usuários Receptores (Instituições):** Organizações do terceiro setor devidamente cadastradas, abrigos, associações de moradores e frentes de assistência social.



- **Impactados Indiretos:** A comunidade em situação de vulnerabilidade, que será a beneficiária final dos alimentos arrecadados.

2.5 Principais etapas necessárias para construir o produto

1. **Levantamento de Requisitos e Modelagem:** Definição do escopo, regras de negócio e modelagem UML (domínio e casos de uso).
2. **Prototipação (Wireframes):** Elaboração de protótipos de baixa fidelidade para validar a interface e a experiência do usuário.
3. **Setup de Infraestrutura:** Configuração do repositório no GitHub para gerenciamento de versões e criação da esteira de Integração e Entrega Contínua (CI/CD).
4. **Desenvolvimento do Back-end:** Construção da API RESTful e modelagem do banco de dados relacional.
5. **Desenvolvimento do Front-end:** Construção das interfaces responsivas e integração assíncrona com a API.
6. **Qualidade e Deploy:** Implementação de casos de teste e implantação da infraestrutura em nuvem na Amazon Web Services (AWS).

2.6 Principais critérios de qualidade para o produto

- **Usabilidade:** Interface web simples, limpa e responsiva (adaptável a telas de celular), permitindo que o fluxo de doação ou reserva ocorra com o mínimo de cliques necessários.
- **Confiabilidade e Segurança:** Implementação de um sistema de autenticação seguro que garanta a identidade dos atores e a integridade das transações, evitando falhas de concorrência (como reservas duplicadas).
- **Escalabilidade e Manutenção:** Código modular, bem documentado e com infraestrutura em nuvem, garantindo disponibilidade contínua e facilitando a adição de futuras funcionalidades.

3. REQUISITOS DO PRODUTO

A tabela a seguir consolida o backlog do produto, classificando os requisitos identificados em Funcionais (RF) e Não Funcionais (RNF), ordenados por prioridade.



ID	Tipo	Descrição Técnica do Requisito	Prioridade
RF01	RF	O sistema deve permitir o cadastro de contas de usuário, diferenciando o perfil entre "Doador" e "Instituição".	Alta
RF02	RF	O sistema deve possuir um mecanismo de autenticação (Login/Logout) validando as credenciais de forma segura.	Alta
RF03	RF	O Doador deve poder registrar um alimento, informando nome, quantidade (ex: Kg, unidades), data de validade e categoria (ex: perecível, não-perecível).	Alta
RF04	RF	A Instituição deve poder visualizar um "feed" contendo todas as doações exclusivas com o status "Disponível".	Alta
RF05	RF	A Instituição deve poder solicitar a reserva de um alimento, o que alterará o status da doação para "Reservado" e exibirá os dados para retirada.	Alta
RF06	RF	O doador deve poder atualizar o status da doação para "Entregue" após a efetivação física da retirada.	Alta
RF07	RF	O sistema deve prover um filtro de pesquisa para que a Instituição busque alimentos por categorias específicas no feed.	Média
RF08	RF	O sistema deve permitir que o doador ou a instituição cancele a operação (doação ou reserva) caso ocorra um imprevisto logístico.	Média
RF09	RF	O sistema deve permitir que o usuário gerencie seu perfil, alterando dados cadastrais como endereço ou senha.	Baixa



4. WIREFRAMES (PROTÓTIPOS DE BAIXA FIDELIDADE)

TELA DE LOGIN

Sistema de Doação

☐ Doador ☐ Instituição



Tela da Instituição (Feed de Alimentos)

Buscar:

ALIMENTOS DISPONÍVEIS:

Ex: 5Kg de Feijão - Mercado XYZ
Validade: 30/03/2026

EX: 30 Pães Franceses - Padaria ABC
Validade: Hoje

Tela do Doador (Cadastrar Alimento)

Nova Doação

Quantidade:

Ex: 20
unid

Validade:

Ex:
25/03/2026

Categoria:



Cadastro de Novo Usuário

Criar Nova Conta

Nome:

CPF/CNPJ:

Endereço:

Email:

Senha:

☐ Sou Doador ☐ Sou Instituição

Cadastrar

Painel / Histórico

Meu Histórico:

- 5kg de Arroz - Entregue dia 10/03
- 20 Pães - Reservado - Aguardando Retirada
- 10 Maças - Ainda Não Reservada

Voltar



A elaboração de wireframes de baixa fidelidade tem o objetivo de validar a arquitetura da informação visual de forma rápida e com baixo custo. O foco nesta etapa é garantir que os fluxos de navegação propostos atendem aos critérios de usabilidade antes do início da codificação das interfaces. Para mapear o fluxo completo do sistema, foram projetadas as seguintes telas principais:

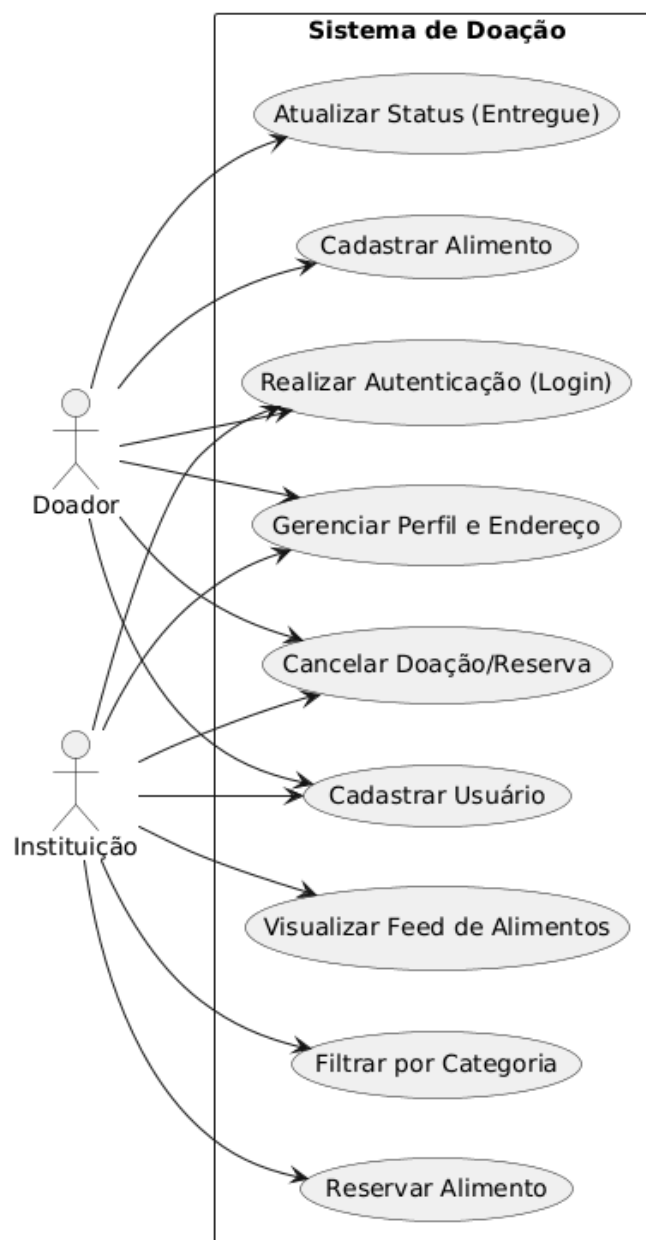
1. **Interface de Autenticação (Login):** Ponto de entrada do sistema, contendo os campos de credenciais (e-mail e senha) e os seletores de perfil para direcionamento correto (Doador ou Instituição).
2. **Interface de Cadastro de Novo Usuário:** Formulário unificado para a entrada de novos atores no sistema, coletando dados essenciais de contato e endereço para viabilizar as futuras coletas logísticas.
3. **Dashboard do Doador (Cadastro de Alimento):** Painel focado na conversão rápida, permitindo que o doador registre um item preenchendo apenas as informações vitais da regra de negócio (nome, quantidade, validade e categoria).
4. **Feed da Instituição (Busca e Reserva):** Tela central da Instituição, projetada no formato de cartões (*cards*) para fácil leitura. Inclui uma barra superior de pesquisa para filtragem ágil e acesso direto ao botão de reserva.
5. **Painel de Histórico e Acompanhamento:** Interface de controle que permite aos usuários rastrear o ciclo de vida das transações no sistema, diferenciando itens concluídos (entregues), em andamento (reservados) e pendentes.

5. MODELAGEM LEVE DO SISTEMA

A modelagem do sistema tem o papel de abstrair a complexidade do software utilizando diagramas UML, padronizando o entendimento das regras de negócio e estruturando o escopo de desenvolvimento técnico para toda a equipe.

5.1 Diagrama de Casos de Uso

O diagrama de casos de uso mapeia as funcionalidades do sistema sob a ótica dos usuários (Atores). Optou-se por um escopo direto, visando a eficiência da aplicação e a entrega de valor ao cliente. Abaixo, apresenta-se a representação visual das interações, seguida pelo detalhamento estrutural de cada caso de uso.





Especificação dos Casos de Uso

A fim de guiar o desenvolvimento do back-end e garantir a correta implementação das regras de negócio, os casos de uso mapeados no diagrama acima são detalhados a seguir:

- **UC1 - Cadastrar Usuário:** Permite que um novo ator crie uma conta na plataforma. O sistema exige a definição do tipo de perfil (Doador ou Instituição) e coleta dados essenciais como Nome/Razão Social, E-mail, Senha e Endereço completo. O endereço é vital para a logística de retirada dos alimentos.
- **UC2 - Realizar Autenticação (Login):** Valida as credenciais (e-mail e senha) do usuário no banco de dados. Em caso de sucesso, o sistema inicia uma sessão segura, direcionando o usuário para o seu respectivo painel (Dashboard de cadastro para Doadores; Feed de busca para Instituições).
- **UC3 - Gerenciar Perfil e Endereço:** Possibilita que o usuário logado atualize suas informações cadastrais a qualquer momento. É um requisito importante caso a Instituição mude de sede ou o doador precise alterar o endereço de coleta.
- **UC4 - Cadastrar Alimento:** Ação central do perfil doador. Permite registrar um novo lote de alimento disponível para doação. O usuário deve informar o título, a quantidade estipulada, a data de validade máxima e classificar o item em uma categoria (ex: perecível, hortifrúti, enlatado). O item entra no sistema com o status inicial de "Disponível".
- **UC5 - Atualizar Status (Entregue):** Após a Instituição reservar o alimento e ir até o local realizar a retirada física, o Doador utiliza esta função para dar "baixa" no sistema, alterando o status da doação de "Reservado" para "Entregue", finalizando o ciclo de vida da transação.
- **UC6 - Cancelar Doação/Reserva:** Fluxo de exceção do sistema. Permite que o doador cancele um anúncio (caso o alimento estrague) ou que a Instituição cancele uma reserva (caso não tenha transporte para ir buscar). O item retorna ao status "Disponível" ou é ocultado.
- **UC7 - Visualizar Feed de Alimentos:** O sistema renderiza uma listagem dinâmica contendo todos os anúncios de alimentos que estão com o status "Disponível" no banco de dados. É a tela principal de navegação da Instituição.
- **UC8 - Filtrar por Categoria:** Função acoplada ao Feed (UC7) que permite refinar os resultados da busca. A Instituição pode selecionar categorias

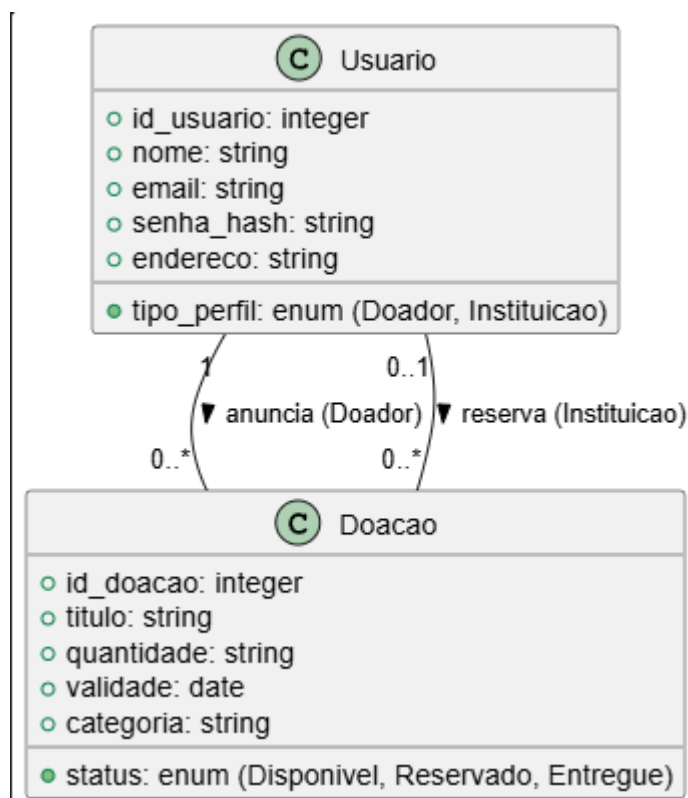


específicas para otimizar sua busca de acordo com a necessidade imediata da ONG.

- **UC9 - Reservar Alimento:** Ação crítica do sistema. A Instituição sinaliza a intenção de buscar aquele alimento. O sistema vincula o ID da Instituição à doação, altera o status do item para "Reservado" e libera os dados de contato e endereço exato do Doador.

5.2 Diagrama de Domínio/Classe

O diagrama de domínio detalha as entidades estruturais fundamentais que serão persistidas no banco de dados e as multiplicidades de suas relações lógicas.



Especificação das Entidades e Relações

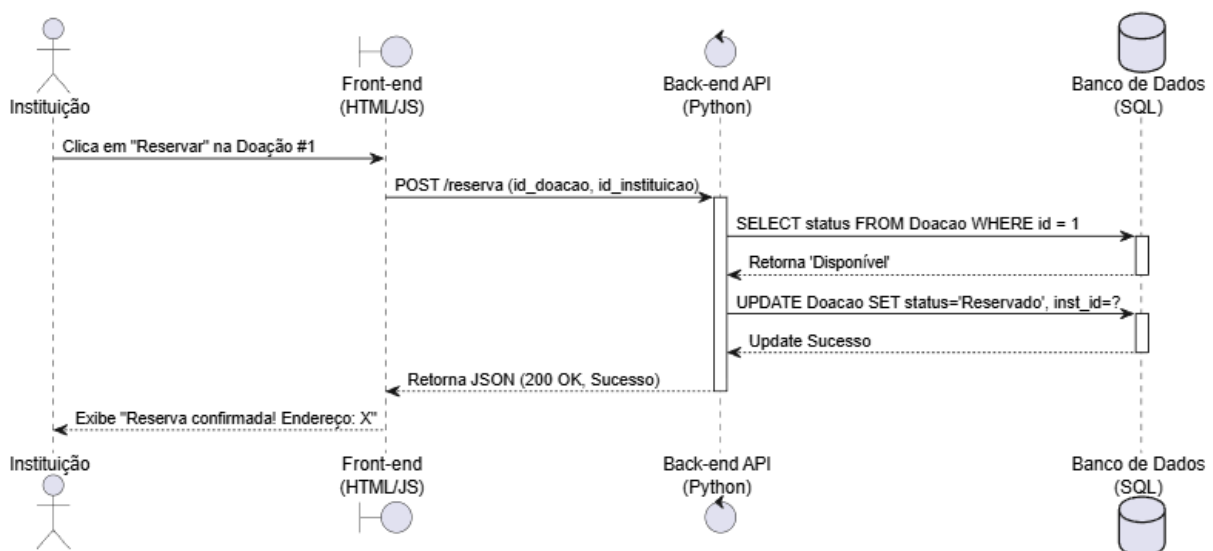
A modelagem de dados foi dividida em duas tabelas principais para manter a normalização e evitar redundância de dados:

- **Entidade Usuário:** Centraliza o acesso ao sistema. Em vez de criar tabelas separadas para "Doadores" e "Instituições", utilizamos o atributo tipo_perfil (enum) para diferenciar os papéis. Isso facilita o sistema de login (UC2), pois a autenticação baterá em uma única tabela, verificando o email e a senha_hash. O endereço fica acoplado ao usuário para facilitar a localização das coletas.

- **Entidade Doacao:** Representa o produto físico. Seus atributos foram escolhidos para sanar as necessidades do Feed (UC7 e UC8). O atributo status é o mais crítico da regra de negócio, pois ele dita a visibilidade do item na plataforma (se está "Disponível", a ONG pode ver se está "Reservado" ou "Entregue", ele some do feed geral).
- **Multiplicidades (Regras de Negócio):**
 - **Anuncia (1 para N):** Um usuário (do tipo Doador) pode anunciar zero ou várias (0..*) Doações, mas uma doação específica pertence a apenas um (1) doador exclusivo.
 - **Reserva (1 para N):** Uma Doação pode ser reservada por no máximo uma (0..1) Instituição (para evitar que duas ONGs busquem a mesma comida), mas uma Instituição pode reservar várias (0..*) Doações no sistema.

5.3 Diagrama de Sequência (Autenticação e Reserva)

Para ilustrar a dinâmica da arquitetura Client-Server, o diagrama abaixo detalha a troca de mensagens na operação mais crítica do sistema: a realização de uma reserva. Ele demonstra a separação de responsabilidades entre o Front-end, a API (Python) e o Banco de Dados.



Especificação do Fluxo de Mensagens

O diagrama acima descreve o fluxo de prevenção de concorrência. Como a plataforma lida com recursos escassos (alimentos), é essencial garantir que duas instituições não reservem o mesmo item simultaneamente. O fluxo ocorre nas seguintes etapas:



1. **Requisição (Front-end):** A Instituição clica em "Reservar". O JavaScript intercepta a ação e dispara uma requisição HTTP do tipo POST para a API, enviando no corpo da mensagem o ID do alimento e o ID da Instituição logada.
2. **Validação de Segurança (API e BD):** Antes de confirmar a reserva, o Back-end em Python faz uma consulta (SELECT) no banco de dados para checar o status atual do alimento. Isso garante que, mesmo que o item estivesse na tela da Instituição, ele não foi reservado por outra pessoa milissegundos antes.
3. **Persistência (Atualização):** Após o banco retornar que o item está "Disponível", a API executa um UPDATE, alterando o status imediatamente para "Reservado" e inserindo o ID da Instituição naquela linha do banco, bloqueando o item.
4. **Resposta e Interface:** O Banco confirma a atualização para a API. A API processa a confirmação e devolve um pacote JSON (Status 200 OK) para o Front-end, que finalmente atualiza a interface para a Instituição, exibindo uma mensagem de sucesso e liberando o endereço de retirada.

6. DESCRIÇÃO DA ARQUITETURA DO SISTEMA E FERRAMENTAS

Para atender de forma estrita aos requisitos mínimos obrigatórios do projeto (separação de front-end e back-end, consumo de API e hospedagem em nuvem) mantendo o foco na velocidade de implementação, qualidade e simplicidade, a equipe definiu uma arquitetura moderna baseada em serviços, adotando o modelo **Client-Server**.

Essa separação garante que a interface do usuário (Client) e as regras de negócio (Server) evoluam de forma independente, o que é ideal para o trabalho em equipe e para a configuração da esteira de Integração Contínua.

6.1 Camada de Front-end (Interface do Usuário)

O Front-end será desenvolvido utilizando tecnologias nativas da web: **HTML5, CSS3 e JavaScript (Vanilla)** puro.

- **Justificativa:** Optou-se por não utilizar *frameworks* pesados (como React ou Angular) para evitar curvas de aprendizado desnecessárias e complexidades na esteira de *build*. O HTML estruturará semanticamente as telas, enquanto o JavaScript assíncrono (através da API nativa `fetch()`) será o responsável por



realizar chamadas HTTP para o nosso Back-end e injetar os dados dinamicamente na tela.

- **Estilização:** Utilizaremos o *framework* CSS **Bootstrap**. Ele permite a criação rápida de interfaces responsivas (adaptáveis para celulares e *desktops*) através de classes pré-prontas, acelerando a prototipação e garantindo uma usabilidade agradável sem a necessidade de escrever milhares de linhas de CSS do zero.

6.2 Camada de Back-end (API e Regras de Negócio)

A API RESTful do sistema, que servirá como o motor lógico da aplicação, será desenvolvida na linguagem de programação Python.

- **Justificativa:** Conforme recomendação da disciplina, Python possui uma sintaxe limpa e direta. Para o desenvolvimento ágil, adotaremos um microframework (como FastAPI ou Flask). Essas ferramentas são minimalistas e permitem a criação de rotas HTTP (endpoints) de forma extremamente concisa. O microframework receberá as requisições em formato JSON enviadas pelo Front-end, processará as regras de negócio e fará a ponte com o banco de dados de maneira segura.

6.3 Camada de Persistência de Dados (Banco de Dados)

O gerenciamento dos dados será feito utilizando a linguagem estruturada **SQL**.

- **Justificativa:** Para a fase inicial de desenvolvimento e testes locais, adotaremos o **SQLite**. Trata-se de um banco de dados relacional embutido que não requer a instalação ou configuração de um servidor local complexo, salvando os dados em um arquivo leve dentro do próprio repositório. O Python utilizará a biblioteca nativa `sqlite3` para executar as consultas (queries). Essa abordagem permite que a equipe foque na modelagem lógica do sistema sem perder tempo com configurações de infraestrutura.

6.4 Ferramentas de Desenvolvimento e DevOps

- **Editor de Código:** Utilizaremos o **Visual Studio Code (VSCode)** devido à sua leveza e integração nativa com extensões essenciais para Python e versionamento.
- **Controle de Versão:** A base de código será hospedada no **GitHub**, cumprindo o requisito obrigatório de atuar como sistema de gerenciamento de versões em nuvem e facilitando a colaboração simultânea dos membros do



grupo.

- **Integração e Entrega Contínua (CI/CD):** A plataforma **GitHub Actions** será utilizada para criar a esteira automatizada. Toda vez que um desenvolvedor enviar (*push*) um novo código para o repositório, um script executará validações automáticas para garantir que a aplicação não contenha erros de sintaxe antes de ser implantada.

6.5 Infraestrutura em Nuvem (AWS)

Para cumprir as diretrizes do projeto e garantir uma entrega profissional, o sistema sairá do ambiente local e será implantado nos recursos de computação em nuvem fornecidos pela **Amazon Web Services (AWS)**. A infraestrutura planejada consistirá em:

- **Amazon EC2 (Elastic Compute Cloud):** O projeto utilizará uma máquina virtual (*instância*) na AWS. Esta instância funcionará como o servidor de produção, hospedando tanto os arquivos estáticos do Front-end quanto o processo Python da nossa API RESTful. Dessa forma, o "Sistema de Doação de Alimentos" estará disponível 24 horas por dia e poderá ser acessado publicamente via internet por qualquer usuário ou instituição, validando o caráter extensionista do projeto.